

This document outlines the architecture for a custom Shopify storefront to sell **Chainmail's** white-label boxed IoT weather sensors and automate the onboarding process for new nodes joining the parametric insurance grid.

1. Product Catalog Architecture

The storefront will offer pre-packaged IoT kits and individual expansion sensors, white-labeled under the **Chainmail Sentinel** brand.

Core Bundles & Hardware Specifications

1. Sentinel Gateway Hub (Model S-100)

- *Based on Ecowitt GW1100/GW1200*
- Central receiver with built-in barometric pressure, indoor temperature, and humidity sensors.
- Connects to local Wi-Fi and pushes encrypted telemetry to the `/api/ingest/ecowitt` endpoint.
- **Price:** \$49.00 USD

1. Wittboy CAT-7 Sensor Array (Model S-700)

- *Based on Ecowitt Wittboy (GW2001)*
- Solar-powered, IP65-rated 7-in-1 outdoor sensor array.
- Measures wind speed (anemometry), wind direction, temperature, humidity, rainfall (hydrology), light, and UV index. Features ultrasonic wind sensor for zero moving parts.
- **Price:** 99.00 USD

1. Sentinel Soil Probe (Model S-WH51)

- *Based on Ecowitt WH51*
- Wireless soil moisture sensor to evaluate regional drought risks and flash flooding saturation.
- **Price:**

4.00 USD

1. Sentinel Smart Valve Actuator (Model S-WFC01)

- *Based on Ecowitt WFC01*

- Smart water timer valve designed to mitigate localized wildfire threat by auto-wetting property grounds when dry soil moisture thresholds trigger parametric alarm states.
- **Price:** \$79.00 USD

2. Automated Network Onboarding Workflow

The Shopify purchase acts as the initial transaction in the node onboarding lifecycle.

```
mermaid
sequenceDiagram
    autonumber
    actor User as Policyholder / Buyer
    participant Shop as Shopify Storefront
    participant Webhook as Shopify Webhook Worker
    participant API as Chainmail API Server
    participant DB as Firestore Database
    participant Device as IoT Sentinel Device

    User->>Shop: Purchase Sentinel Kit (S-100 + S-700)
    Shop-->>User: Order Confirmation & Shipping Label Generated
    Shop->>Webhook: Event: order/paid (Payload contains email, address)
    Webhook->>API: POST /api/onboarding/provision
    activate API
    API->>API: Generate secure device key & pre-provision propertyId
    API->>DB: Write property & pre-provisioned sensors as PENDING_ACTIVATION
    API-->>Webhook: Provisioning Success
    deactivate API
    Webhook-->>User: Onboarding Email with Device Credentials & Claim QR Code
    User->>Device: Mount hardware & connect to home Wi-Fi
```

```
Device—>API: POST /api/ingest/ecowitt (First Telemetry Ping)

activate API

API—>API: Authenticate device with secure pre-provisioned key

API—>DB: Update status to VERIFIED_NODE & Activate Parametric Coverage

API-->>Device: Acknowledge (HTTP 200)

deactivate API

API—>User: App Push Notification: "Node Active. Coverage Live!"

---
```

3. API Provisioning Endpoint Specifications

When a checkout completes, Shopify sends an `order/paid` webhook to our backend API server:

Endpoint: `POST /api/onboarding/provision`

Access Control: HMAC Signature verified using Shopify API Webhook Shared Secret.

Request Payload (from Shopify):

```
{
  "type": "json",
  "data": {
    "order_id": 987654321,
    "email": "customer@example.com",
    "shipping_address": {
      "address1": "12665 Julington Oaks Dr",
      "city": "Jacksonville",
      "province": "FL",
      "zip": "32223",
      "country": "US"
    },
    "line_items": [
```

```
{  
  
  "sku": "SENTINEL-KIT-A",  
  
  "quantity": 1,  
  
  "name": "Sentinel Gateway & Wittboy Array"  
}  
  
]  
}
```

Provisioning Logic:

1. **Address Validation:** Call Google Address Validation API to verify CASS status and retrieve exact rooftop coordinates (lat / lng).
2. **Database Record Creation:**

- Create a new document in /properties:

```
typescript  
{  
  
  ownerEmail: "customer@example.com",  
  
  address: "12665 Julington Oaks Dr, Jacksonville, FL 32223",  
  
  lat: 30.1421936,  
  
  lng: -81.6157784,  
  
  riskScore: 72.4, // Calculated by Underwriting Agent based on coordinates  
  
  status: "PENDING_ACTIVATION",  
  
  createdAt: serverTimestamp()  
}
```

- Create a document in /properties/{propertyId}/sensors/Anemometry:

```
typescript  
{
```

```
type: "Anemometry",

verificationStatus: "UNVERIFIED",

hardwareType: "Ecowitt",

value: 0

}
```

1. **Secret Generation:** Generate a unique password/secret token for the gateway and email it as a QR code to the user.

4. UI/UX Storefront Integration

The Shopify storefront theme will match Chainmail's core brand design system: **Cobalt & Gold / Slate & Gold**.

Theme Design System

- **Primary Background:** #030712 (Slate Black)
- **Secondary Cards:** rgba(17, 24, 39, 0.7) (Translucent Dark Blue)
- **Accents:** #D4AF37 (Metallic Gold)
- **Fonts:** Outfit (Headings) and Inter (Body)

Custom Liquid Elements

- **Network Node Counter:** A live ticker in the header showing: ACTIVE_NODES: 1,482 | TOTAL_COVERAGE_LOCKED: \$34,891,000`.
- **Map Twin Preview:** An interactive WebGL globe on the landing page showing glowing gold dots representing active coverage sensors in real time.
- **onboarding-tab widget:** Built-in section on the product page showing the simple "Plug, Scan, Insure" 3-step setup guide.

3. Job Code Testing Targets

- **CEO:** Focuses on multi-signature sequences, treasury ledgers, and cash asset parity.
- **COO:** Evaluates checkout webhooks, hardware delivery tracking, and device MAC pairing validations.
- **CRO / Legal:** Verifies Zero-Knowledge privacy proofs and automated state reports.

- **Actuaries:** Monitors coordinate drift triggers and risk pricing correctness.

,500 payout cap) just so we can get their sensor data and help protect the community.

The Claims Adjuster Agent: When a disaster hits, this AI looks at satellite photos to verify the damage. If it's confident, it pays the customer instantly. If it isn't sure, it sends the claim to a human to review—it is strictly programmed **never** to automatically deny a claim.

The Vigilance Oracle Agent: This AI constantly watches the weather channel. If it sees a Category 5 hurricane coming, it flags the map and readies our bank accounts to pay people before they even ask.

The Quant Brain Actuary Agent: This AI watches our bank account. If we take on too much risk in one state, it automatically buys "reinsurance" (backup insurance for us) to keep the company financially safe.

Your Day-to-Day Responsibilities

As a Product Owner, you will:

- **Monitor the Vision:** Look at the Enterprise Dashboard. Are the AI agents making decisions that align with the BallPark Initiative?
- **Prioritize the Backlog:** Work with the Product Manager to decide which neighborhood of our "city" needs improvement next.
- **Understand the Costs:** The Enterprise dashboard shows you exactly how many fractions of a cent the AI is spending to make decisions. You balance the cost of the AI against the value it provides.

You don't need to know how to code the AI. You just need to know what the AI **should** be doing to help our customers. Welcome to the team! ,500."*

Here is how you handle it:

- Explain that because their house is in an **Extreme Risk Zone** (like right on the beach), our Underwriting AI cannot offer them half a million dollars right now.
- However, explain that we **did not reject them**. We offered them the **IoT Baseline Policy**. This gives them ,500 of emergency cash, and in return, they install a weather sensor on their house. This helps us gather data to protect the rest of their neighborhood!

Need Help?

If the screen turns white, or the map stops loading, do not panic! Just take a screenshot and send it to your Product Manager with the Jira ticket label UI-BUG.

Welcome to Chainmail! We are glad you're here.

DATA PRIVACY & ACCOUNT DELETION (GUIDELINE 5.1.1)

Apple absolutely mandates that any app allowing account creation *must* also provide an in-app option to completely delete the account and all associated data.

Action Item: Build an automated "Delete Account" API route in the Vercel backend.

- **App Tracking Transparency (ATT):** If we use analytics that track across third-party apps, we must implement the ATT prompt.
- **Minimum Functionality:** The app cannot just be a "wrapped website" (Guideline 4.2). It must utilize native iOS features (e.g., Push Notifications, FaceID/TouchID for secure login, local caching).

3. Critical Google Play Requirements (Android)

Google Play has recently tightened restrictions on testing and financial apps.

20-TESTER RULE

New Google Play developer accounts must run a closed test with a minimum of 20 opted-in testers for at least 14 continuous days before you can apply for production access.

Action Item: We must recruit 20 beta testers internally or via the Vigilance Grid network well before launch.

FINANCIAL FEATURES POLICY

Google requires a prominent "Financial Features Declaration" during submission. If the app facilitates insurance or digital settlement payouts, we must provide links to our regulatory compliance documentation and operating licenses in our Google Play Console declaration.

- **Data Safety Section:** We must accurately declare exactly what data is collected (e.g., location data from Vigilance Grid IoT integration, financial info) and whether it is encrypted in transit.

4. Next Steps for the Development Team

1. **Architecture:** We must decide whether to wrap the Vercel app using a framework like Capacitor/Ionic, or rebuild the frontend natively using React Native.
2. **Privacy Policy URL:** A live, hosted Privacy Policy must be created on the public-facing website.
3. **Support URL:** A dedicated support and contact page must be established for user inquiries.